What we built: architecture and design choices

Dynamic Team, Amazon ML Challenge 2026. Every figure is from our own code, logs or data.

1. The pipeline at a glance

For each Source 2 or 3 record we find the one Source 1 business it copies, or decide it has none. Counts are for the test set; scores are out-of-fold on the training set.

Step	What runs	Pairs after it	True pairs kept	Offline score	Time
Normalise	Rules and tables, no model				3 min
Blocking	Sparse TF-IDF top-k inside each state	266 M (27 per record)	99.17%		24 min
Stage 1, prune	LightGBM, 300 trees, 26 features	9.73 M (5.6 per entity)	98.35%		55 min
Stage 2, match	2 LightGBM averaged, 66 features	same pairs, scored		0.9836	95 min
Stage 3, differences	LightGBM, 600 trees	same pairs, re-scored		0.9878	25 min
Stage 4, collective	LightGBM, 1,200 trees, open records only	2.39 M open pairs		0.9894	37 min
Text models	4 cross-encoders on the open pairs			0.9915	1.5 h GPU
Decision	Threshold 0.75, margin 0.1, three rules	5.85 M matches	97.50%	0.9915	7 min

The pruned set after stage 1 is what we submitted as `candidate_pairs.tsv`. Only the text models need a GPU.

2. Normalisation

What it does. Lower-case ASCII; legal forms (Inc, LLC, Pvt Ltd, SARL) mapped to one spelling and kept in their own field; digits inside words turned back into letters (Onc0logy); IDs, phone numbers and websites stripped from the name (the website kept separately); “doing business as” names split off; street abbreviations expanded; house numbers pulled out, with “305-307”, “305 1/2” and “305B” read as one number; Indic-script words transliterated with a dictionary of 1,347 words that we learned by aligning matched training pairs (a word is kept if seen at least twice and at least half of the time; unknown words fall back to character-by-character transliteration).

Where the state comes from

The data has no state column. There are only four columns: ID, name, address, country. We extract the state from the address text. We split the address at its commas and look each piece up in a table of state names and codes for that country (US states, Indian states, French regions and departments). The lookup ignores case, accents and punctuation, so “PA”, “Pa.” and “Pennsylvania” all give the same state. Indian state names in native script are mapped with the learned dictionary.

Typos in a state name are not corrected. The lookup is exact after cleaning. A misspelt state is simply not recognised, the record gets no state, and it is then searched against the whole country instead of one state. So a typo costs time for that record, not recall. 95.3% of true-copy records have a recognisable state.

Why “Indiana, PA” needed special handling

Indiana is a town in Pennsylvania and also the name of a state. The address “…, Indiana, PA” therefore contains two pieces that look like a state. If we took the first one, the record would be searched among businesses in the state of Indiana and its true business in Pennsylvania would never be found. The same happens with “Delaware, OH” and “Washington, DC”. Our rule when several pieces look like a state: prefer a name that is not also a common place name; prefer a two-letter code over a full name; prefer the later piece over the earlier. So “PA” wins, “Indiana” is kept as part of the address (it is the town), and Indiana is stored as an **alternative state** that is also searched, in case the rule chose wrong.

How well it works. Of true-copy records with a state, 99.47% have the same state as their business, 0.03% are rescued by the alternative state, and 0.50% sit in another state.

3. Blocking (candidate generation)

What it does. Each record becomes a sparse vector of weighted keys: name words; name words without vowels; the first four letters of each word; the first six letters of the joined name (for websites); the legal form at low weight; address words; house numbers. Weights are IDF, computed per country without labels. The name part and the address part are each scaled to length 1, and the score is 0.6 × name similarity + 0.4 × address similarity. An exact sparse top-k product returns three rankings per record, which we merge: top 20 by the combined score, top 10 by name only, top 10 by address only. A record is searched in its own state, its alternative state, and neighbouring states learned from the labels (two pairs in India, such as Telangana and Andhra Pradesh). A record with no state is searched in the whole country.

Choice	Why	Evidence
Partition by state, not postal code or city	There are no postal codes. City names are unreliable: true copies of one business say “Belle Meade” and “Nashville”	Recall rose from 99.06% to 99.22% (US) and 98.15% to 98.80% (India) when we moved from one index per country to states
TF-IDF, not embeddings	Exact, CPU only, keys can be inspected	14,000 records per second on a laptop. Dense retrieval was not tested
Three rankings	A record with no address, or with a changed name, is still found by the other ranking	With an address, recall is 99.96% (US), 99.57% (India)

4. The four model stages

All are LightGBM trained with log loss. We do not cap tree depth; we cap leaves per tree.

Stage 1, pruning. 300 trees, 63 leaves. 26 cheap features: retrieval scores, ranks and gaps; three string ratios; flags such as same state, same first house number, address empty. It keeps at most 2 candidates per record and drops any below 0.01. *Why 2:* keeping 8 raises the recall ceiling from 98.35% to 98.87%, but in that run only 453 of 7.29 M correct matches came from a candidate beyond the top two.

Stage 2, matcher. 66 features: the 26 above plus 40 more (several name and address similarities, how the house numbers relate, how many records claim the same business, the stage-1 probability). Two models are averaged: 2,000 trees with 127 leaves, and 3,000 trees with 255 leaves. *Why two:* averaging a smaller and a larger model is more stable than either alone.

Stage 3, differences. 600 trees, 127 leaves. Input: the stage-2 probability, rank and gap, plus features that describe how the two records differ: the signed house-number difference and whether the length changed; words left over, swapped in, added or misspelt; which letter of a word changed. *Why:* a sibling and a typo have the same similarity; only the kind of difference separates them. The ten offsets are not coded in; the model learns them.

Stage 4, collective. 1,200 trees, 63 leaves, 150 features. It runs only on records still open (stage-3 probability between 0.001 and 0.999). New features: how many other records, already matched to the same business with probability 0.9 or more, share this record’s unusual number, address or name; whether same-source copies agree with its number; spelling cues from the raw names; and the text-model scores. *Why only confident records count:* otherwise two siblings could vouch for each other.

Why separate stages. Each needs the previous one’s predictions: stage 4 cannot count confident matches before stage 3 has scored everything. It also keeps expensive features off the 79% of records that are already settled.

Why LightGBM. It scores 170,000 pairs a second on a laptop, the evidence is mostly numeric, and its features stay inspectable, which is how we found the siblings.

5. Text models

Four public MIT checkpoints fine-tuned as cross-encoders: MiniLM (33 M parameters, 3 epochs), XLM-R base (278 M), XLM-R large (560 M) and DeBERTa-v3 base (184 M), 2 epochs each. Input: the raw name and address of both records, up to 128 tokens. They are trained only on the 0.9 M training pairs that stage 3 was unsure about, with the same folds, and give stage 4 three features each: probability, margin over the record’s other candidate, and how many other records they assign to the same business. *Why only uncertain pairs:* that is where they are needed, and it keeps GPU time at 1.5 hours. *Why not a transformer for everything:* 266 M pairs, and the decisive evidence (number arithmetic, other records) is not in the text of one pair. Log loss on the uncertain pairs: best transformer alone 0.241, stage 4 without text 0.218, both together 0.150.

6. Decision and the three rules

Each record keeps only its best candidate. It is accepted if the probability is at least 0.75 and 0.1 ahead of the second-best. Then:

- **Word-swap veto.** Cancel a match if a name word was replaced by a different real name word, wherever it stands.
- **France rule.** A French match needs two votes: stage 4 without text models accepts it, and stage 4 with them gives at least 0.75.
- **Sibling-group rule (US, India).** Drop a match at 0.98 or below whose unusual house number is shared with records of the same business that we rejected with confidence. It changes 374 matches.

Why 0.75: a wrong match costs about 2.7 times what a right one earns, so a match pays off only above about 0.73; a grid search on the real metric gave 0.75.

7. Validation

Two folds, split by the true business, so no business is in both. Every model in every stage is cross-fitted, so every training pair has a score from a model that never saw that business; on test the two fold models are averaged. We compute the organisers' exact macro F0.5 on all 2.21 M training businesses, and choose thresholds on a simulated test mix in which distractor records are repeated until they are 40%.

8. What we did not do

- No separate fold for hyper-parameter tuning; a few variants were compared on the same out-of-fold score.
- No ranking loss, no dense retrieval, no city-level blocking: none of these was tested.
- No fuzzy correction of misspelt state names, and no handling of a business that moved address.
- The search index is held in memory per country; the sharded version for billions of records is not built.
- The France rule and the sibling-group rule were chosen with the public leaderboard, not offline.